Yahtzee clásico con estrategia de Montecarlo

Documento técnico

Santiago Andres Valencia Duque

IU Digital de Antioquia

Ingeniería de Software y Datos

Módulo 1 · Actividad dinámica 2 M1

Medellín, Colombia

25 de octubre de 2026

Repositorio: github.com/andresvd95/Simulacion

1. Introducción

Este proyecto desarrolla una aplicación de Yahtzee para dos jugadores y una herramienta visual para observar su comportamiento. El juego utiliza cinco dados, 13 turnos por jugador y hasta tres lanzamientos por turno. Cada dado es independiente y tiene seis resultados equiprobables.

El trabajo se organizó con la metodología **USA, MODIFICA y CREA**. Primero se comprendió y reutilizó la solución inicial; después se mejoraron sus reglas, estrategias y formas de ejecución; finalmente se creó un frontend interactivo para ejecutar partidas y comparar experimentos. La referencia de las reglas es el manual oficial de Hasbro [1].

2. Proceso USA, MODIFICA y CREA

2.1 USA: comprender y reutilizar

En la etapa **USA** se tomó como punto de partida la solución inicial del juego. Se revisó cómo se lanzaban los dados, cómo se calculaban las puntuaciones, cómo se llenaba la tarjeta y cómo se gestionaban los turnos. La estructura existente se reutilizó para conservar la lógica central y separar las responsabilidades: lanzamiento, puntuación, tablero, estrategias y simulación.

2.2 MODIFICA: mejorar la solución inicial

En la etapa **MODIFICA** se incorporaron las reglas completas del Yahtzee: categorías superiores e inferiores, bono superior de 35 puntos, Yahtzees adicionales de 100 puntos y reglas Joker. También se añadieron dos estrategias automáticas. `greedy` decide mirando solo la mano actual; Montecarlo prueba continuaciones aleatorias para estimar qué dados conviene conservar.

Además, se modificó el programa para ejecutar muchas partidas automáticamente. Se agregaron los parámetros de número de partidas, muestras por candidato y semilla aleatoria. La semilla permite repetir el mismo experimento y comparar resultados bajo las mismas condiciones.

2.3 CREA: construir una herramienta visual

En la etapa **CREA** se construyó un servidor web local con Python y una interfaz HTML, CSS y JavaScript. El frontend permite elegir el número de partidas, las muestras y la semilla. Tiene dos acciones principales:

- **Jugar partida demo:** ejecuta una partida Montecarlo contra greedy y muestra los dados finales, el marcador y las categorías utilizadas.

- **Simular lote:** ejecuta muchas partidas y muestra victorias, empates, puntajes medios, mínimos y máximos mediante barras y tablas.

Durante el cálculo, la interfaz muestra un loader con mensajes como “Lanzando dados”, “Buscando la mejor retención”, “Evaluando decisiones Montecarlo” y “Comparando estrategias”. Así se puede observar qué está haciendo el programa mientras procesa la simulación.

3. Diseño y reglas del juego

La solución separa lanzamiento de dados, cálculo de puntos, manejo del tablero y estrategias de decisión. Cada jugador dispone de una tarjeta con las seis categorías superiores y siete inferiores. Al cerrar un turno debe ocupar una casilla libre, incluso si vale cero. La suma superior de al menos 63 recibe 35 puntos. Un Yahtzee adicional aporta 100 puntos si la casilla Yahtzee contiene 50; las reglas Joker obligan a usar primero la casilla superior correspondiente, luego una inferior disponible y por último otra superior con cero [1].

Categoría	Puntuación
Unos a seises	Suma de los dados del valor elegido
Trío / cuarteto	Suma de los cinco dados si hay al menos 3 / 4 iguales
Full	25 por tres iguales y dos iguales
Escalera corta / larga	30 por cuatro consecutivos / 40 por cinco consecutivos
Yahtzee / azar	50 por cinco iguales / suma de los dados

4. Método de Montecarlo

4.1 Cómo toma decisiones el jugador Montecarlo

Para cada categoría libre, la estrategia construye un conjunto de dados a conservar según su objetivo: dados del mismo número, del grupo más frecuente, valores únicos de una escalera o valores altos para azar. También evalúa conservar los cinco dados. Para cada candidato genera N continuaciones con las caras equiprobables; si quedan dos lanzamientos, usa una regla de conservación orientada a la misma categoría entre el segundo y el tercero. Calcula la media de `puntos finales − costo de reserva de la casilla` y elige el candidato de mayor media. Al terminar los lanzamientos, anota la mejor categoría legal con esa misma función de utilidad.

Los costos de reserva son heurísticos explícitos del código: evitan gastar una categoría valiosa por pocos puntos. Por tanto, la media Montecarlo aproxima el valor de esos candidatos y esa política de continuación, no la estrategia matemáticamente óptima. La estrategia de comparación, `greedy`, usa la mano actual para elegir objetivo y no muestrea continuaciones.

```
para cada turno y jugador:
    lanzar 5 dados
    hasta 2 veces:
        si es Montecarlo: estimar utilidad media de cada candidato
        si es greedy: conservar dados según la mejor categoría actual
        relanzar dados no conservados, salvo si se conservan los 5
    elegir categoría legal y registrar puntos
sumar casillas, bono superior y bonos Yahtzee
```

4.2 Implementación del experimento

Se implementó en Python con su biblioteca estándar. `random.Random(seed)` permite repetir una ejecución y `randint(1, 6)` modela los dados uniformes. El modo `play` ofrece jugadores humanos, Montecarlo o greedy; `simulate` ejecuta partidas completas y exporta JSON. El experimento de referencia usa 100 partidas, 24 muestras por candidato y semilla 2026. Montecarlo ocupa el puesto de jugador 1 y greedy el de jugador 2.

```
python Act1/yahtzee.py simulate --games 100 --samples 24 --seed 2026 --output
Act1/resultados.json
```

5. Resultados y Teoría de los Grandes Números

La ejecución reproducible produjo los siguientes valores, disponibles también en `resultados.json`.

Medida	Montecarlo	Greedy
Victorias	53	46
Puntuación media	215,38	214,17
Puntuación mínima	130	123
Puntuación máxima	398	490

Hubo un empate. La diferencia de medias fue de 1,21 puntos y la diferencia de victorias fue de siete partidas. El resultado muestra una pequeña ventaja de Montecarlo en esta ejecución, pero no significa que gane siempre: el máximo de greedy fue superior y cada partida está influenciada por el azar.

5.1 Justificación de la Teoría de los Grandes Números

La Teoría de los Grandes Números indica que, al repetir muchas veces un experimento aleatorio independiente, el promedio observado tiende a acercarse al valor esperado. En este proyecto, los dados se generan con resultados uniformes entre 1 y 6 y las partidas se repiten con las mismas reglas.

La evidencia se observa al comparar promedios y no una sola partida: después de 100 partidas, Montecarlo obtuvo 215,38 puntos de media y greedy 214,17. Los resultados extremos fueron 130 y 398 para Montecarlo, y 123 y 490 para greedy, lo que confirma que existe variabilidad individual. Sin embargo, al agrupar las partidas, los valores medios son mucho más representativos que cualquiera de esos extremos.

Por tanto, el experimento cumple la teoría como evidencia empírica: las repeticiones permiten que los casos favorables y desfavorables se compensen y que el promedio sea más estable. Para observar una convergencia todavía más clara se podrían ejecutar 1.000 o 10.000 partidas y repetir el experimento con varias semillas. Las 100 partidas y una sola semilla no constituyen una demostración matemática absoluta, pero sí muestran el comportamiento esperado de la ley.

6. Conclusiones

Se completó un juego de dos participantes con reglas de anotación clásicas, selección de dados mediante Montecarlo y ejecución masiva reproducible. El muestreo permite estimar el resultado de decisiones de conservación sin enumerar todas las secuencias futuras de dados. En el ensayo registrado, Montecarlo obtuvo una ventaja pequeña frente a la estrategia greedy; es un resultado descriptivo del experimento, no una garantía de mejor desempeño.

Como mejora posterior se podría comparar varias semillas, medir la variabilidad y ajustar o sustituir los costos de reserva por una función de valor del tablero completo. La aplicación ya permite cambiar el tamaño del experimento y las muestras por candidato desde la línea de comandos.

7. Referencias bibliográficas

1. Hasbro. *Yahtzee: instrucciones oficiales*. https://www.hasbro.com/common/instruct/yahtzee.pdf. Consultado el 21 de octubre de 2026.

2. Python Software Foundation. *random — Generate pseudo-random numbers*. https://docs.python.org/3/library/random.html. Consultado el 21 de octubre de 2026.